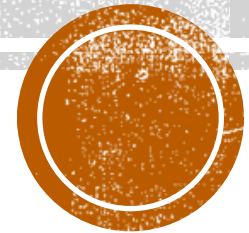




Module: Développer en Front-end

Manipuler les propriétés et gérer les états



Filière: Développement digital – option web full stack

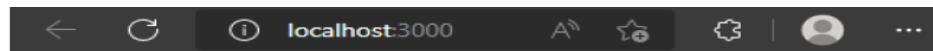
Réalisé par: ELMAUDNI

Manipulation des props dans un composant(Rappel)

- les composants sont réutilisables, si on est dans la situation où on souhaite afficher le même composant plusieurs fois mais avec des informations différentes. React utilise un Object props qui va nous permettre de passer les informations nécessaires au composant, ce qui rend le composant dynamique.
- Exemple:

```
function Presentation(props)
{ console.log(props)
  return ( <div >
    <h2>Salut {props.nom} {props.prenom}</h2>
    </div> ) }
```

```
function App(){ return(
  <div>
    <Presentation nom="Rami" prenom="Ahmed"/>
    <Presentation nom="Kamali" prenom="Ali" />
    <Presentation nom="Fahmi" prenom="Khalid"/>
  </div>
) }
root.render(<App/>)
```



Salut Rami Ahmed

Salut Kamali Ali

Salut Fahmi Khalid

```
▶ {nom: 'Rami', prenom: 'Ahmed'}
▶ {nom: 'Kamali', prenom: 'Ali'}
▶ {nom: 'Fahmi', prenom: 'Khalid'}
```

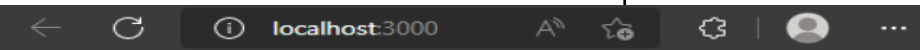
Manipulation des props dans un composant créé via une classe

- Exemple:

```
class Presentation extends React.Component{  
  render(){  
    console.log(this.props)  
    return (  
      <div >  
        <h2>Salut {this.props.nom}  
        {this.props.prenom}</h2>  
        <hr/>  
      </div>) }}  

```

```
function App(){ return(  
  <div>  
    <Presentation nom="Rami" prenom="Ahmed">  
    </Presentation >  
    <Presentation nom="Kamali" prenom="Ali" />  
    <Presentation nom="Fahmi" prenom="Khalid"/>  
  </div>  
)}  
root.render(<App/>)
```



Salut Rami Ahmed

Salut Kamali Ali

Salut Fahmi Khalid

```
▶ {nom: 'Rami', prenom: 'Ahmed'}  
▶ {nom: 'Kamali', prenom: 'Ali'}  
▶ {nom: 'Fahmi', prenom: 'Khalid'}
```

Passer dynamique contenu a un composant

- l'élément Presentation contient un élément enfant entre la balise ouvrante et fermante.
- Pour récupérer l'élément enfant en utilise la propriété children : **{props.children}**
- Exemple:

```
function Presentation(props){  
  console.log(props)  
  return (  
    <div >  
      {props.children}  
  
      <h2>Salut {props.nom} {props.prenom}</h2>  
      <hr/>  
    </div>  
  ) }
```

```
function App() {  
  return (  
    <div>  
      <Presentation nom="Rami" prenom="Ahmed">  
        <p>ce ci est un children props</p>  
      </Presentation>  
      <Presentation nom="Kamali" prenom="Ali">  
        <button>quitter</button>  
      </Presentation>  
    </div>  
  ); root.render(<App/>)
```

Salut Rami Ahmed

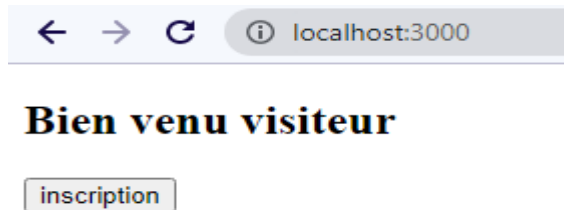
ce ci est un children props

Salut Kamali Ali

quitter

Utiliser l'objet state pour mettre à jour un composant (Classe composant)

- State est un objet pouvant être mis à jour qui peut être utilisé pour contenir les données et contrôler le comportement du composant. Seuls les composants de classe peuvent avoir un état, pas les composants fonctionnels. Lorsque l'état est modifié, React restitue automatiquement le composant qui provoque une mise à jour d'affichage.
- Considérons que nous souhaitons écrire un composant Message simple qui permet d'afficher le rendu suivant:



Si l'utilisateur clique sur le bouton inscription



le click sur le bouton:

- le message:
Bien Venu visiteur → votre inscription est effectuée
- le bouton:
inscription → merci.

D'où il faut avoir deux variables message et btnMessage dont les valeurs vont être modifiées après la clique sur le bouton, cette modification doit provoquer un rafraichissement d'affichage → nécessité d'utiliser **l'objet State**.

```
class Message extends React.Component{  
  constructor(){  
    super()  
    this.state={message:"Bien venu  
visiteur",btnMessage:"inscription"}  
  }  
}
```

```
inscription(){  
  this.setState({message:"votre inscription est  
effectuée",btnMessage:"merci"})  
}  
render(){  
  return(<div>  
    <h2>{this.state.message}</h2>  
    <button onClick={()=>this.inscription()}  
    >{this.state.btnMessage}</button>  
    <button onClick={()=>this.inscription()}  
    >valider</button>  
    </div>))}
```

1. Création de l'objet state qui continent les deux propriétés message et btnMessage ,les deux propriétés sont initialisées.
2. Mise à jour de state : La modification des valeurs des propriétés se fait via la fonction **this.setState()**
 - La fonction setState modifie les propriétés puis provoque un appel de la méthode render(), c'est ainsi que l'interface utilisateur est mise à jour avec les nouvelles valeurs de **{this.state.message}** et **{this.state.btnMessage}**

Règles de state.

- Il ne faut pas modifier l'état state directement
- ~~this.state.message="message 1"~~
- ~~this.state.btnMessage="message 2"~~
- en effet cette modification directe change l'état mais elle ne provoque pas l'appel de la méthode **render()**, d'où l'interface utilisateur n'affiche pas les nouvelles valeurs des propriétés de l'état .

state est un java script objet contenant des données relative au composant

la mise à jour de 'state' sur le composant provoque le rendu instantané du composant

state doit être initialisé quand le composant est créé

state doit être mise à jour uniquement par la fonction **'setState'**

Utiliser state pour mettre à jour un composant fonctionnel

Utilisation des hooks pour gérer l'état de composant créé par fonction:

Les Hooks ont été ajoutés à React dans la version 16.8.

Pour utiliser le Hook useState il faut l'importer

```
import React, { useState } from "react";
function AutreMessage() {
  const [message, setMessage] = useState("Bien venu visiteur");
  const [btnMessage, setBtnMessage] = useState("inscription");
  function inscription() {
    setMessage("votre inscription est effectuée");
    setBtnMessage("merci");
  }

  return (
    <div>
      <h2>{message}</h2>
      <button onClick={() => inscription()}>{btnMessage}</button>
    </div>
  );
}
```

```
const stateMessage=useState("Bien venu visiteur")  
const message=stateMessage[0]  
const setMessage=stateMessage[1]
```



ces deux écritures sont identique
on a utilisé dans la deuxième écriture (Array
destructeur JS ES6)

```
const [message, setMessage] = useState("Bien venu visiteur")
```

```
const [message, setMessage] = useState("Bien venu visiteur")
```

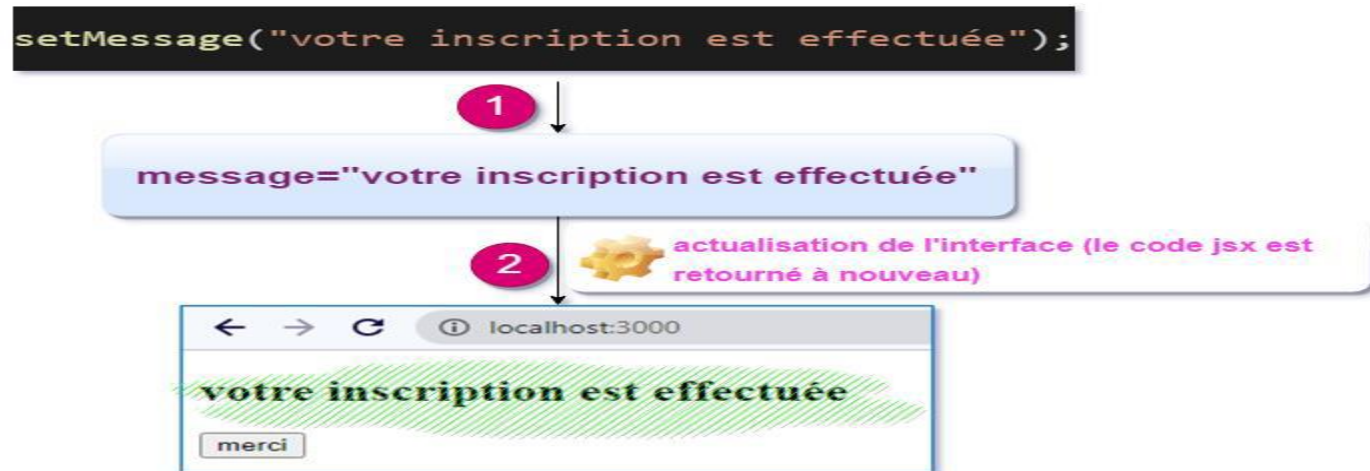


valeur
reactive



setter
méthode

- Le hook `useState` est une fonction fournie par REACT qui vous permet d'ajouter les fonctionnalités de React dans votre composant fonctionnel.
- Le hook `useState` est une fonction qui retourne un Array contenant deux éléments
- Le premier élément c'est la propriété `message` que l'on peut utiliser dans le composant fonctionnel, la propriété `message` est initialisée par l'argument de la fonction `useState`, dans ce cas `message` est initialisé par "Bien venu visiteur".
- Le deuxième élément retourné c'est la fonction `setMessage`. la méthode `setMessage` permet de changer la valeur de la propriété `message` puis lance un rafraîchissement de user interface.



Remarque : Attention Il ne faut pas changer directement les propriétés d'états !!!!

```
let [message, setMessage] = useState("Bien venu visiteur");
let [btnMessage, setBtnMessage] = useState("inscription");

function inscription() {
  message="votre inscription est effectuée";
  btnMessage="merci";
  console.log(message);
  console.log(btnMessage);
}
```

- On a changé const par let
- Puis dans la méthode inscription on a changé directement les propriétés d'Etats
- On remarque que après le click l'interface n a pas changé
- Le rendu du console :

```
votre inscription est effectuée
merci
```

Exercice d'application

- Réalisation d'un compteur



- Le click sur le bouton incrémenter incrémente le compteur
- Le click sur le bouton décrémenter décrément le compteur
- La valeur du compteur doit être affiché